

Analysis: Ingredient Optimization

The problem is talking about two types of events:

- Delivery: when an amount of leaves is added to the stock with a specific expiry time.
- Order: when an amount U of leaves is removed from the stock if we have enough.

In order to keep track of the amount of stock we have at some point of time, we can loop through the events in a sorted order by delivery/order time. This approach helps us avoid using invalid delivery for a certain order (For example: a delivery at time 3 is not valid to be used in an order at time 2).

For example: If we have a delivery at time 1, 3, 5 and order at time 2, 3, 6, we will process them in the following order:

[Delivery1, Order2, Delivery3, Order3, Delivery5, Order6]

Note that if an order and delivery have the same time stamp, we need to process the delivery first.

This can be done by using two pointers (one for the deliveries and one for the orders) and pick the smaller delivery/order time first. Another alternative is adding all of them into one array and sort it.

Test Set 1

For Test Set 1, no Thai basil leaf spoils before an order comes. So, we don't really care about the expiry time. In this case, we can represent the stock by a single number and loop through the delivery/order events. Whenever we have a:

- Delivery: the stock is increased by the delivered amount.
- Order: the stock is decreased by U amount only if we can fulfill the order.

```
if currentEvent is Delivery:
    stockAmount += currentEvent.deliveredAmount
else if currentEvent is Order:
    if stockAmount >= U:
        stockAmount -= U
        fulfilledOrders += 1
```

Time Complexity

Since the deliveries and orders are given in ascending order, we only need to go through each event. Therefore the time complexity equals $O(N + D)$.

Test Set 2

For Test Set 2, we need to take the expiry time for each delivery into consideration. The key observation is when we have multiple leaves to choose from, we must always choose the U

leaves that spoil first. This way we increase the chances to fulfill more future orders. Therefore, having a sorted list of deliveries by expiry time will help optimize using the leaves.

We need to store a sorted list which contains detailed information about deliveries. For each delivery, we need to store the delivery time, expiry time, and amount of Thai leaves. Whenever we have a:

- Delivery: the delivery information is added to the deliveries list in the correct position so that all the deliveries that spoil before it are placed before it on the list. Also, the amount of the stock is increased by the delivered amount.
- Order: we check our stock amount. If it's greater than U , then we can fulfil this order. In this case, we must use the U non-spoiled leaves that will spoil first in the future and remove all the used leaves from the list of deliveries.

Note that we continuously need to remove the spoiled leaves before processing the delivery/order event.

The approach explained above is using the insertion sorting algorithm to add the delivery in the correct order which gives a time complexity $O(D^2)$. A better approach is to use a Min-Heap, such that the earliest leaves to spoil are always found on the top of the heap. In this case, adding a delivery to its right place in the heap will be $O(\log D)$. While, removing the used leaves from the heap should be $O(\log D)$ as well.

Time Complexity

In case of using insertion sort, the total complexity of inserting the deliveries in their right place will be the complexity of insertion sort, $O(D^2)$. And for fulfilling the orders, we iterate through $O(N)$ orders and visit and remove at most $O(D)$ deliveries from the list. So the total complexity is $O(D^2 + N)$.

In case of using a Min-Heap, we insert to the heap for each delivery, and remove a delivery from the top of the heap when it is used up or spoiled, so there are totally $O(D)$ insertions and removals on the heap. The top of the heap is checked for each order and whenever a previous top is removed, so there are totally $O(D + N)$ checks. Since the operation on a heap is $O(\log D)$, the total complexity $O((D + N) \log D)$.

Analysis: Interesting Outing

There is exactly one way to get from one interesting sight to another using these transportation methods without using any transportation method more than once. So the set of interesting places and modes of transportation is an unrooted [tree](#).

Test Set 1

For Test Set 1, the limits are small enough to calculate visit time for every order in which we can visit the sights. Each of the order corresponds to one [permutation](#) of the numbers 1 to N , and there are $N!$ such permutations.

Once we have fixed the order we will visit the sights, we simply need to compute the distance from the first sight to the second, the second to the third, and so on until we reach the end point. The distance between two sights can be computed in many different ways. For example, with [Floyd-Warshall algorithm](#) or running [BFS](#) or [DFS](#) N times. Note that you can precompute all of these distances beforehand rather than re-computing them every time. Each of the mentioned algorithm has a polynomial runtime. So, time to precompute is $O(N^2)$ or $O(N^3)$ for each permutation. Thus, the total time complexity is $O(N \times N!)$, which is sufficient for Test Set 1.

Test Set 2

For Test Set 2, we need a more efficient approach. From this point onwards, we will call the set of sights and the transportation methods in the input *tree*, where the sights are its [nodes](#) and the transportation methods are its [edges](#).

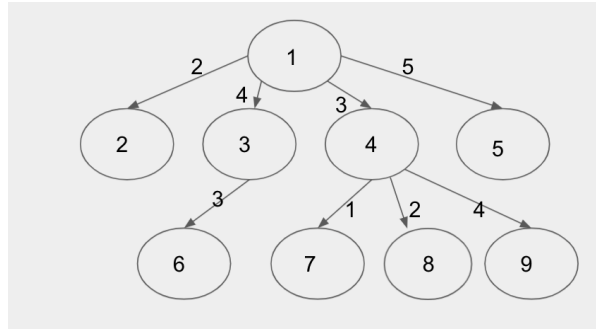
Let us consider a fixed starting node S and the tree as rooted with S as its root. There are N choices for S and we will try them all. Our solution traverses the rooted tree top-down, while trying to minimise the total cost. While traversing, the path will go only once into each subtree. Going back to any subtree after visiting it once will never be optimal. The upper bound of the cost of visiting a rooted tree is $2 \times \sum(C_i)$ (think about traversing in DFS fashion, starting from and ending at the root). If any path goes more than once into any subtree, it will result additional cost.

One interesting observation is that the cost of visiting a subtree rooted at any node depends on which of the child subtrees will be visited last. For example, if we have four child subtrees of the root, and we have decided which subtree to visit last, the total cost will be the same, regardless of which order we decide to visit the first three subtrees. In order to visit all of the nodes, we must return to the root every time we finish traversing each subtree, except the last one, to be able to move to the next one.

However, this observation is not true for all subtrees. When we are at a subtree that doesn't have the starting point as the root, the cost of visiting the subtree depends on another factor, that is, *whether this subtree was chosen as the last one to be visited from its parent or not*.

If it was not the last one, then we need to go back to the parent, so the cost will be constant in the rooted tree, no matter what order we visit the subtrees.

If it was the last one, then the traversal can end at any node, it doesn't need to go back to the parent. In that case, the ordering of visiting subtrees will matter.



In the example picture of a graph above, the cost of visiting all the places if we start from node 1 will depend on which of the four child subtrees we are visiting last. Then, when we visit any child subtree, we also need to consider whether that subtree is the last one visited from its parent 1, or not.

From the above observations, we can formulate a recurrence to find the optimal cost. Let's denote the cost function as $F(x)$ for a subtree rooted at x , and it has been selected as the last child subtree during the traversal from its parent, and $F_1(x)$, when it has not been selected as the last one. Then we can have,

$$F(x) = \min_{y \in Ch(x)} \left(F(y) + E(x, y) + \sum_{z \in Ch(x) \setminus \{y\}} (2 \times E(x, z) + F_1(z)) \right)$$

Here,

- $Ch(x)$ are the set of nodes that are children of x ,
- y is the selected last one to be visited from its parent x , and
- $E(x, y)$ is the cost of travelling between x and y .

$F_1(x)$ is a constant function for a given weighted tree and fixed root. Since the starting and ending nodes are the same, all the edge costs in the subtree rooted at x will be included in the total cost twice. We can precompute these values for a fixed root.

We can try all the nodes as a starting point. For each of them, we can calculate the cost using the recurrence. At each node x , we will have at most $O(\text{degree}(x))$ options to choose the last child subtree. However, the total number of choices are $\sum (\text{degree}(\mathbf{N}))$, which is $O(\mathbf{N})$ for a tree.

By using memorisation, we can make sure the function $F(x)$ and $F_1(x)$ is being calculated only once per node.

We can avoid running the inner loop to calculate the sum $\sum_{z \in Ch(x) \setminus \{y\}} (2 \times E(x, z) + F_1(z))$ for every choice of y by calculating the sum $\sum_{z \in Ch(x)} (2 \times E(x, z) + F_1(z))$ at the beginning of the loop, and for every y , subtracting $2 \times E(x, y) + F_1(y)$ from it.

These techniques enable us to execute the calculations for each choice in constant time. We have $O(\mathbf{N})$ options for starting points. Hence, the total time complexity becomes $O(\mathbf{N}^2)$, which is sufficient for Test Set 2.

Alternate solution

There's another approach to solve this problem. Let's think about any walk that visits all the places, has starting place $\mathbf{S}$, and ending place $\mathbf{T}$. Let's consider extending this path to also walk back from $\mathbf{T}$ to $\mathbf{S}$ at the end. The length of such enclosed path is exactly $\text{distance}(\mathbf{S}, \mathbf{T})$ longer than the length of the walk. But note that all enclosed paths that touch every node require a cost

of at least $2 \times \sum(C_i)$ since we have to go "down" and "up" every edge. Thus, the optimal enclosed path is to use $2 \times \sum(C_i)$. Then to find the cheapest walk, we want to subtract off the largest distance(**S**, **T**), which is exactly the longest path in the tree.

The longest path in a tree can be found in $O(N)$ time using two cleverly selected DFSs. Alternatively, we can simply run N DFSs, one from each node, to find the distance between every pair of nodes, then take the maximum, giving us an $O(N^2)$ algorithm. Hence, the total time complexity becomes $O(N)$ or $O(N^2)$, which is sufficient for Test Set 2.

Analysis: Inventor Outlasting

Since the available actions of any game state don't depend on whose turn it is, we can classify each game state into two outcomes: The next player (the one whose turn it is) wins (denoted as an N -position), or the previous player wins (denoted as a P -position). For example:

- N -position means: If it's Izabella's turn, then Izabella is going to win; if it's Olga's turn, then Olga is going to win.
- P -position means: If it's Izabella's turn, then Izabella is going to lose; if it's Olga's turn, then Olga is going to lose.

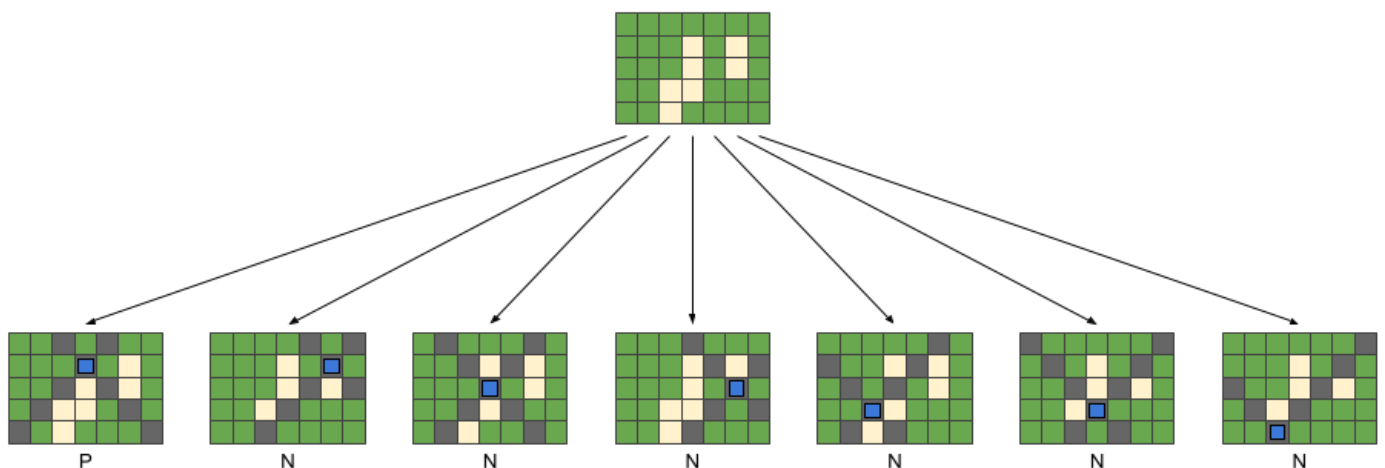
This kind of game is called an [impartial game](#).

With the game rules from the problem statement, to decide if a given game state s is an N -position or a P -position, we have the following recursion rules:

- If there exists a spot where building an attraction on it can turn the current game state s into a P -position. In other words, forcing the opponent to lose. Then s is classified as an N -position.
- Otherwise, s is classified as a P -position because s doesn't have any available spots to build an attraction, or building an attraction on any spot will always turn the current game state into an N -position. In other words, letting the opponent win.

The goal of this problem is to find out how many spots there are on the given initial game state that the player can build an attraction on it and turn the game state into a P -position state.

Take Sample Case #1 as example, the seven different next states and their position types are



Since only one of those states is a P -position, the answer to Case #1 is 1.

Test Set 1

For this test set, since we only have at most 10 spots available for new attractions, we can simply use brute force search with the above recursion rules to decide whether a given game state is N -position or P -position.

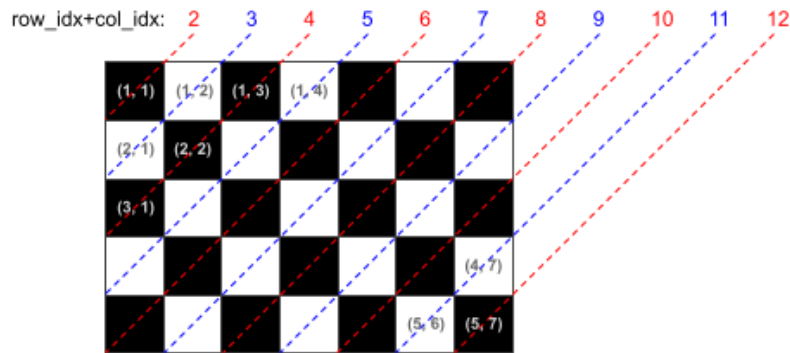
The game tree size and the state-space complexity of this game both have size $O(S!)$, where S represents the number of spots available to build an attraction on the initial game state. For each state, in addition to the recursion for next states, we also need $O(\min(\mathbf{R}, \mathbf{C}))$ time to mark the new advertisement signs on the board. Therefore, the brute force search solution to this problem has time complexity $O(S! \times \min(\mathbf{R}, \mathbf{C}))$.

In this brute force search, using [dynamic programming](#) to store every seen state won't help decrease the time complexity because the size of the dynamic programming table we will end up with is still up to $O(S!)$.

Test Set 2

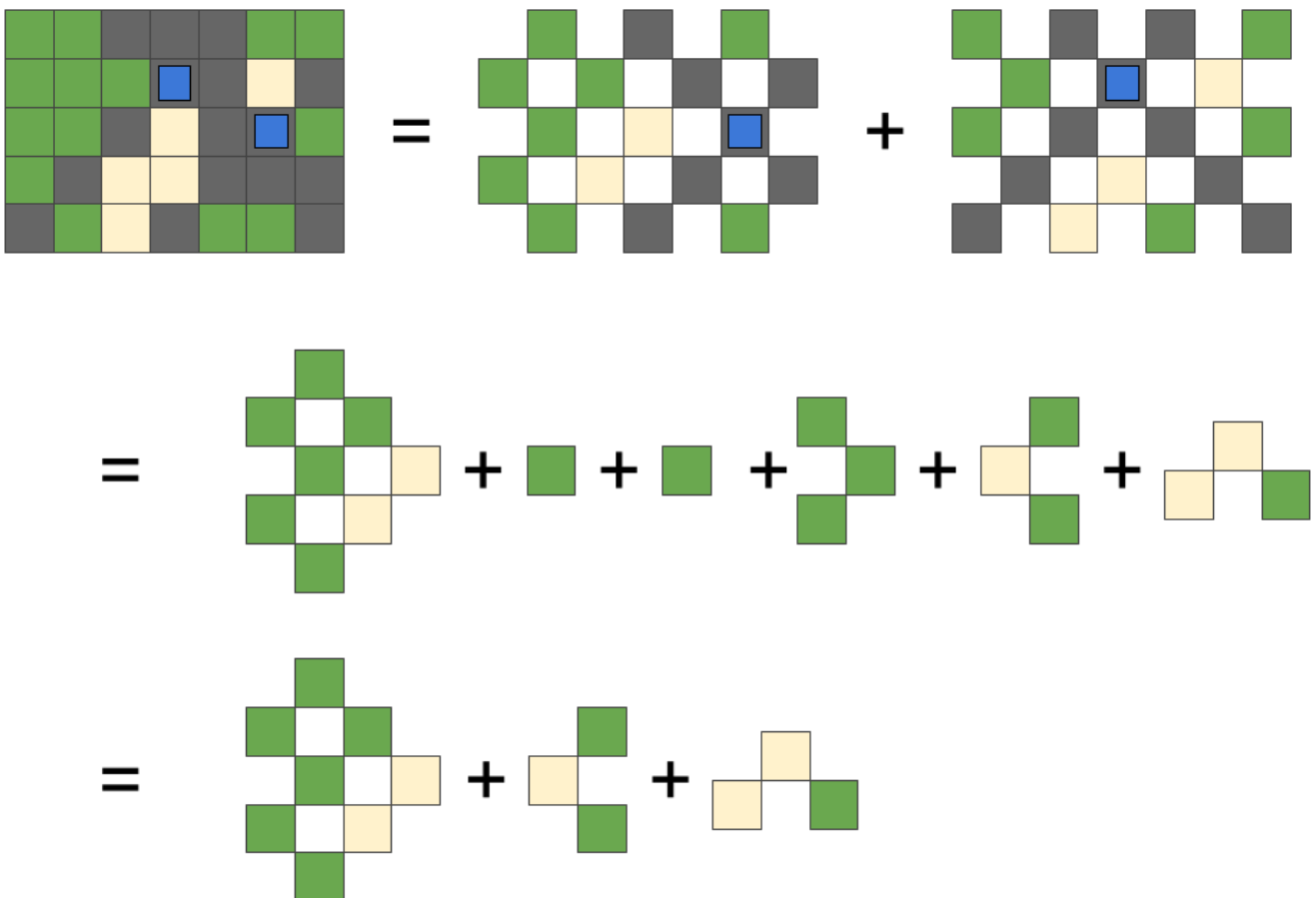
Since the sign builders always extend in the diagonal direction like [chess bishops](#), the cells on the game board can actually be seen as two independent parts: the black cells and the white cells in a chessboard shown

below.



Formally, from each attraction, only cells that have the same $\text{row_idx} + \text{col_idx}$ parity can be affected. See the chess board above, the black cells have even $\text{row_idx} + \text{col_idx}$, while the white cells have odd $\text{row_idx} + \text{col_idx}$.

Furthermore, since the sign builders only extend until they reach an occupied cell, the areas that were previously divided by other extending signs are also independent. And the independent areas with no available spot to build can simply be ignored.



For example, the top left is a board with 2 attractions built on (2, 4) and (3, 6). This board is equal to 6 independent "sub-games" shown above. And from observation, we can see only 3 of them have available spots, and each of them will be completely occupied in one move. Therefore, there are exactly 3 moves remaining to finish the game no matter how the players play. So we can easily figure out that this example is an N -position (the next player wins).

To formalize the calculation steps we demonstrated, we need the [Sprague-Grundy theorem](#). According to the Sprague-Grundy theorem, if we have an impartial game in which the players take turns to play exactly one move and the player who cannot move loses, we can assign each game state a value (called the SG -value). Each game-state's SG -value only depends on the SG -value of the game-states we can reach with exactly one move.

Specifically, for any game state s , we have:

- $SG(s) = 0$ if s doesn't have any available spots to build an attraction.
- Otherwise, let $\{p_1, p_2, \dots, p_n\}$ be all possible game states after building exactly one attraction at each of the available spots on s . Then we have that $SG(s) = \text{mex}(\{SG(p_1), SG(p_2), \dots, SG(p_n)\})$, where $\text{mex}(X)$ means the minimum non-negative integer that doesn't exist in the set X .

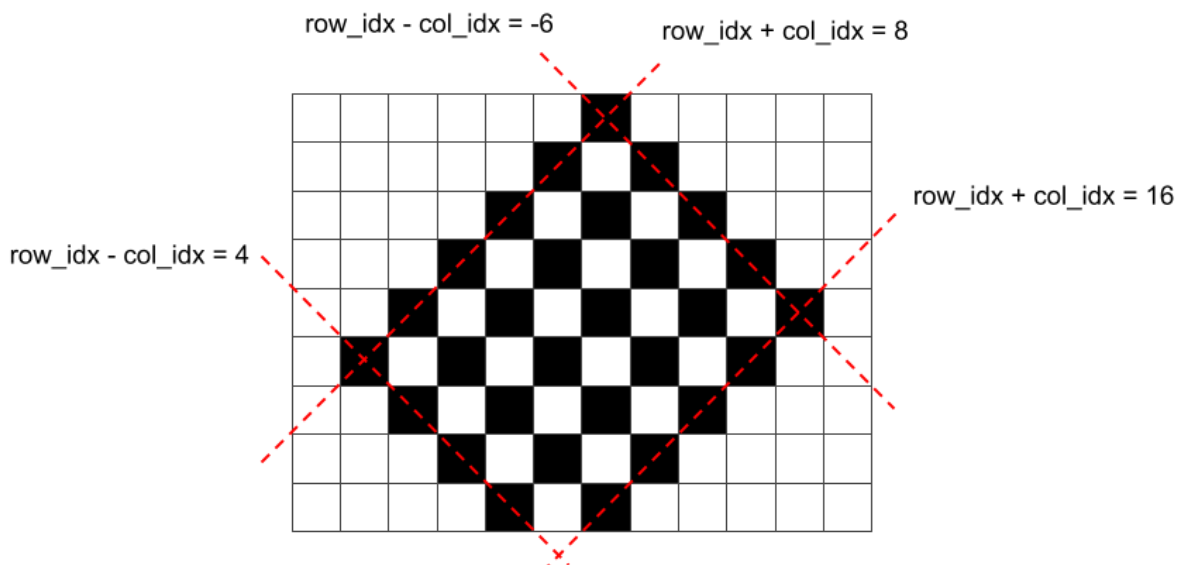
If a state s is split into several independent sub-games $\{q_1, q_2, \dots, q_k\}$, the SG -value of s can be calculated as

$$SG(s) = SG(\{q_1, q_2, \dots, q_k\}) = SG(q_1) \oplus SG(q_2) \oplus \dots \oplus SG(q_k),$$

where $\oplus$ is the [binary xor function](#).

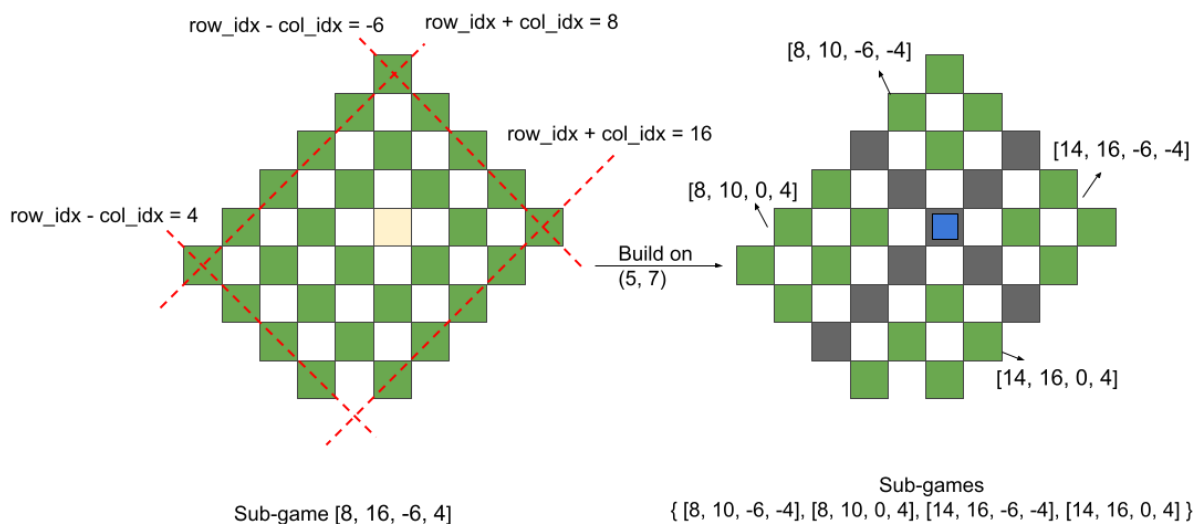
Finally, if $SG(s) = 0$, then s is a P -position; otherwise s is an N -position. So the goal of this problem is to find out how many spots on the given initial game state the player can build an attraction on and turn the game state into SG -value = 0.

The recursive function to calculate the SG -value of a given sub-game can be implemented as:



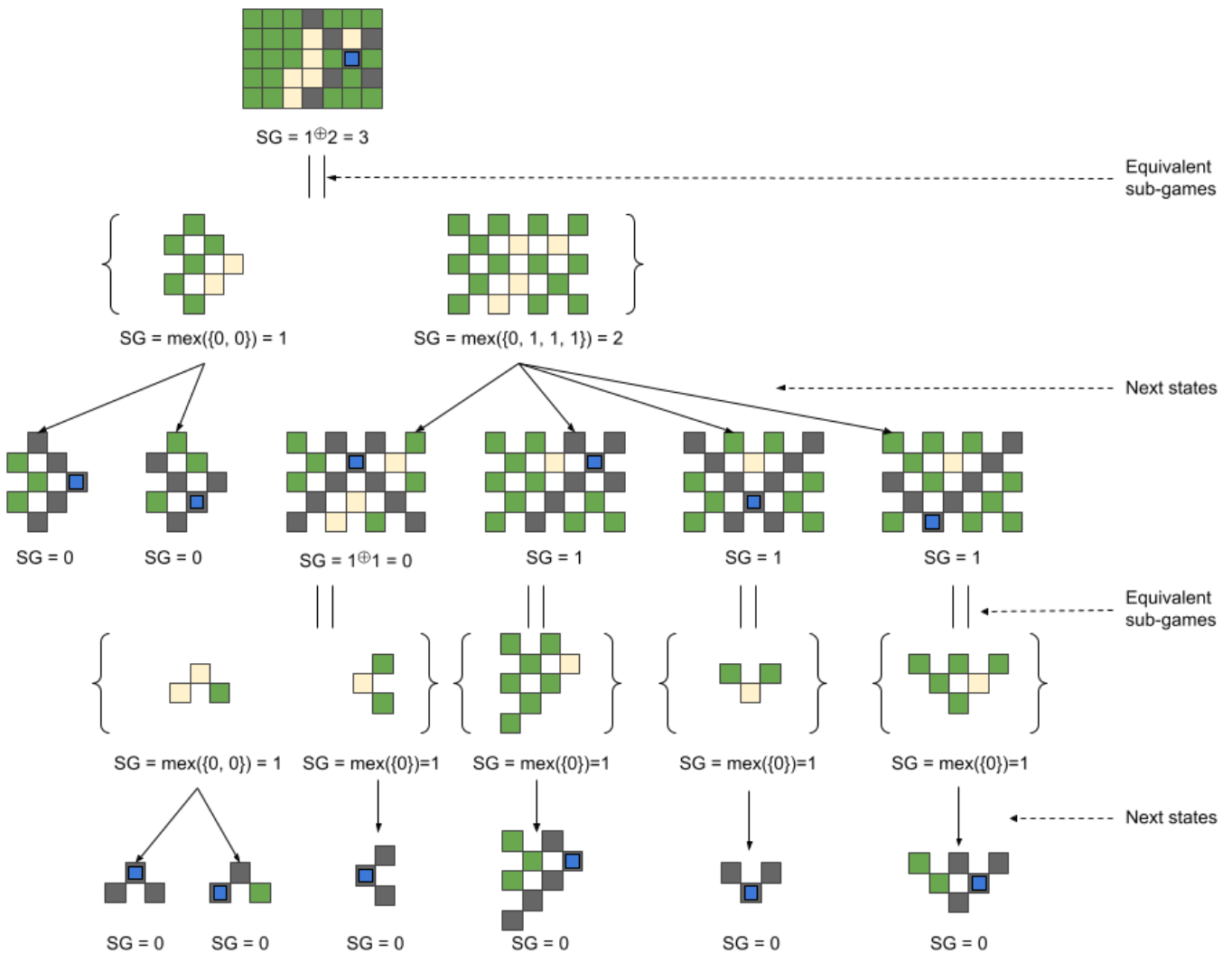
The sub-game marked as black cells is represented as [8, 16, -6, 4].

- a sub-game can be represented by a tuple [row_idx_plus_col_idx_begin, row_idx_plus_col_idx_end, row_idx_minus_col_idx_begin, row_idx_minus_col_idx_end], like the example shown above,
- for each available spots in this range, put an attraction on it and calculate the new SG -value. Then the SG -value of this sub-game is calculated by applying mex to them. And



- when an attraction is put on a spot $[row_idx, col_idx]$, the current sub-game $[row_idx_plus_col_idx_begin, row_idx_plus_col_idx_end, row_idx_minus_col_idx_begin, row_idx_minus_col_idx_end]$ is divided into four new sub-games
 - $[row_idx_plus_col_idx_begin, row_idx + col_idx - 2, row_idx_minus_col_idx_begin, row_idx - col_idx - 2]$,
 - $[row_idx_plus_col_idx_begin, row_idx + col_idx - 2, row_idx - col_idx + 2, row_idx_minus_col_idx_end]$,
 - $[row_idx + col_idx + 2, row_idx_plus_col_idx_end, row_idx_minus_col_idx_begin, row_idx - col_idx - 2]$,
 - $[row_idx + col_idx + 2, row_idx_plus_col_idx_end, row_idx - col_idx + 2, row_idx_minus_col_idx_end]$,
 as the example above, and the SG -value of the original sub-game with the new attraction on it is calculated as XOR of the SG -values of those four new sub-games

Now we know how to use the Sprague-Grundy theorem. Take Sample Case #1 as an example, what will happen if Izabella puts her attraction at third row and sixth column?



We can see that putting the first attraction on this cell will give an SG -value of 3. This means that Olga will have a winning move from this position, which is not good for Izabella!

Note that we can use [dynamic programming](#) to store the SG -value of each seen sub-game. In this way, the state-space complexity of this search is $O(\mathbf{R}^2 \times \mathbf{C}^2)$. And when calculating the SG -value of each sub-game, we need to visit all $O(S)$ available spots in it to enumerate the next states. When adding an attraction, we don't need to actually mark those $O(\min(\mathbf{R}, \mathbf{C}))$ advertisement signs, we only need to use constant time to split the sub-games and calculate XOR from their results. Therefore, the total time complexity of this solution is $O(S \times \mathbf{R}^2 \times \mathbf{C}^2)$. And since S is $O(\mathbf{R} \times \mathbf{C})$, the time complexity can also be written as $O(\mathbf{R}^3 \times \mathbf{C}^3)$.

Analysis: Inversions Organize

Test Set 1

Since $N \leq 2$, there will be a maximum of 16 elements in the matrix. We can simply brute force this by trying every possible combination and checking which installation that satisfies the organizational goal (top N rows has the same number of `1`s as the bottom N rows, and left N columns has the same number of `1`s as the right N columns) involves the minimum number of letter switches. Since there are at most 2^{16} total possible combinations, this is fast enough for Test Set 1.

Test Set 2

To solve Test Set 2, we can first split the matrix into quadrants. Let A , B , C , and D be the number of `1`s in each quadrant, in the following order:

```
A B
C D
```

In Sample Case #1, we would split it as follows:

```
1 1  0 0
0 0  0 1
```

```
1 1  1 1
0 0  0 1
```

Here, $A = 2$, $B = 1$, $C = 2$, and $D = 3$.

Similarly, let A' , B' , C' , and D' be the number of `1`s on each quadrant of the output, in the same order as before. Then, we need A' , B' , C' , and D' such that $A' + B' = C' + D'$ and $A' + C' = B' + D'$.

Adding the two equations, we obtain $A' = D'$, and replacing that in either equation, we obtain $B' = C'$. Notice that having $A' = C'$ and $B' = D'$ are also sufficient conditions for the original equations. Therefore, we can solve the equivalent problem of minimizing the letter touches to get $A' = D'$ and $B' = C'$. We can see now that fulfilling $A' = D'$ and $B' = C'$ are independent problems.

Then, to find the minimum number of letter changes, we need to find the sum of the differences between each pair of sets, A and D , and B and C (to achieve equalization, we can greedily switch that amount of `1`s to `0`s from the side that has more `1`s).

To implement this idea, we iterate through the matrix and keep track of the counts per quadrant, A , B , C , D , and return the summation of absolute differences: $|A - D| + |C - B|$.

This algorithm runs in $O(N^2)$ time, since we are iterating through the matrix.